EXPERIMENT 7

Aim: Build an Artificial Neural Network (ANN) using Keras/TensorFlow

1. Dataset Source

Dataset used: **Churn Modelling Dataset (Kaggle)**
Link: https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling

2. Dataset Description

The dataset contains bank customer information used to predict whether a customer will leave the bank (churn) or not.

- **Type:** Classification Dataset
- **Instances:** 10,000 records
- **Features:** 14 columns

Key Features:

- CreditScore – Customer's credit score
- Geography – Country (France, Spain, Germany)
- Gender – Male/Female
- Age – Customer age
- Tenure – Number of years with bank
- Balance – Account balance
- NumOfProducts – Number of bank products used
- HasCrCard – Credit card availability
- IsActiveMember – Active membership status
- EstimatedSalary – Salary

Target Variable:

- **Exited (0/1)**
  - 0 → Customer stays
  - 1 → Customer leaves

Data Characteristics:

- Contains categorical + numerical features
- Requires encoding and scaling
- Slightly **imbalanced dataset**

3. Mathematical Formulation of CNN

Artificial Neural Network consists of interconnected neurons organized in layers.

Neuron Output:

$$z = \sum (w_i x_i) + b$$

$$a = f(z)$$

Where:

- w_i = weights
- x_i = inputs
- b = bias
- f = activation function

Activation Functions:

ReLU:

$$f(x) = \max(0, x)$$

Sigmoid:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Loss Function (Binary Classification):

$$Loss = -\frac{1}{N} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Backpropagation:

- Uses gradient descent to update weights
- Minimizes loss function

4. Algorithm Limitations

- Requires large dataset for better performance
- Sensitive to hyperparameters
- Risk of overfitting
- Black-box model (low interpretability)
- Struggles with imbalanced data (like churn dataset)

5. Methodology / Workflow

Steps Followed

1. **Data Loading**
    - Load dataset using pandas
2. **Data Preprocessing**
    - Remove unnecessary columns (RowNumber, CustomerId, Surname)
    - Encode categorical data:
        - Gender → Label Encoding
        - Geography → One-hot encoding
3. **Train-Test Split**
    - 80% training, 20% testing
4. **Feature Scaling**
    - Standardization using StandardScaler
5. **Model Building**
    - Input Layer
    - Hidden Layer 1 (16 neurons, ReLU)
    - Hidden Layer 2 (8 neurons, ReLU)
    - Output Layer (1 neuron, Sigmoid)
6. **Model Compilation**
    - Optimizer: Adam
    - Loss: Binary Crossentropy
7. **Training**
    - Epochs: 50
    - Batch Size: 32
8. **Prediction**
    - Predict churn probabilities
9. **Evaluation**
    - Accuracy + Confusion Matrix

6. Performance Analysis

Results Obtained:

- **Test Accuracy ≈ 86%**
- Training Accuracy ≈ 87%
- Validation Accuracy ≈ 85–86%

Graph Analysis:

- Training and validation accuracy curves are close
- Indicates **good generalization**
- No overfitting observed

Confusion Matrix:

```
[[1530   77]
 [ 201  192]]
```

Interpretation:

- Model predicts **non-churn customers very well**
- Slightly lower accuracy for churn cases
- Reason: **Class imbalance**

7. Hyperparameter Tuning

Parameters Used:

- Hidden Layers: 2
- Neurons: 16, 8
- Epochs: 50
- Batch Size: 32
- Optimizer: Adam

Tuning Impact:

Parameter	Effect
More neurons	Better learning but risk of overfitting

More epochs	Higher accuracy but longer training
Learning rate	Controls convergence
Batch size	Affects speed and stability

Possible Improvements:

- Add Dropout layer
- Use more hidden layers
- Apply class balancing techniques (SMOTE)
- Use early stopping
- Tune learning rate

8. Code and Output

```
# ================================

# 1. Import Libraries

# ================================

import numpy as np

import pandas as pd

import tensorflow as tf



from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler



# ================================
```

```
# 2. Load Dataset
# ================================
df = pd.read_csv('/content/Churn_Modelling.csv')

# Display dataset
print(df.head())

# ================================
# 3. Data Preprocessing
# ================================

# Drop unnecessary columns
df = df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1)

# Encode categorical variables
label_encoder = LabelEncoder()

df['Gender'] = label_encoder.fit_transform(df['Gender'])

# One-hot encoding for Geography
df = pd.get_dummies(df, columns=['Geography'], drop_first=True)

# Split features and target
```

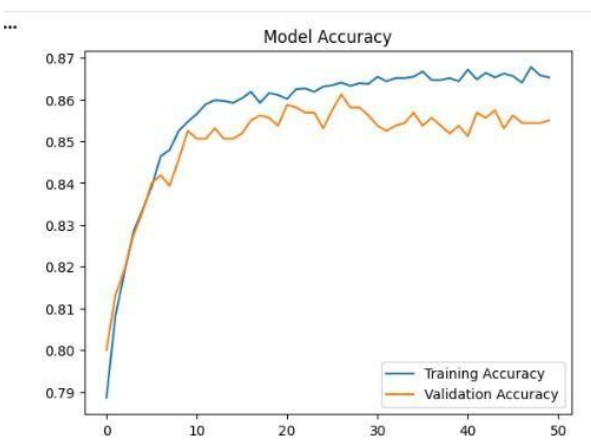
```
X = df.drop('Exited', axis=1)

y = df['Exited']


# ================================

# 4. Train-Test Split

# ================================

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


# ================================

# 5. Feature Scaling

# ================================

sc = StandardScaler()


X_train = sc.fit_transform(X_train)

X_test = sc.transform(X_test)

# ================================

# 6. Build ANN Model

# ================================

model = tf.keras.models.Sequential()


# Input + Hidden Layers
```

```
model.add(tf.keras.layers.Dense(units=16, activation='relu',
input_dim=X_train.shape[1]))

model.add(tf.keras.layers.Dense(units=8, activation='relu'))



# Output Layer

model.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))

# ================================

# 7. Compile Model

# ================================

model.compile(

    optimizer='adam',

    loss='binary_crossentropy',

    metrics=['accuracy']

)

# ================================

# 8. Train Model

# ================================

history = model.fit(

    X_train, y_train,

    batch_size=32,

    epochs=50,

    validation_split=0.2

)

# ================================
```

```
# 9. Evaluate Model
# ================================
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)


# ================================
# 10. Predictions
# ================================
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5)


# ================================
# 11. Confusion Matrix
# ================================
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Model Accuracy")
plt.show()
```

```
...    RowNumber  CustomerId   Surname  CreditScore Geography  Gender  Age  \
0          1    15634602  Hargrave          619    France  Female   42
1          2    15647311      Hill          608     Spain  Female   41
2          3    15619304      Onio          502    France  Female   42
3          4    15701354      Boni          699    France  Female   39
4          5    15737888  Mitchell          850     Spain  Female   43

   Tenure    Balance  NumOfProducts  HasCrCard  IsActiveMember  \
0       2       0.00              1          1               1
1       1   83807.86              1          0               1
2       8  159660.80              3          1               0
3       1       0.00              2          0               0
4       2  125510.82              1          1               1

   EstimatedSalary  Exited
0        101348.88       1
1        112542.58       0
2        113931.57       1
3         93826.63       0
4         79084.10       0
Epoch 1/50
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
200/200 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.7886 - loss: 0.4907 - val_accuracy: 0.8000 - val_loss: 0.4517
Epoch 2/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8083 - loss: 0.4383 - val_accuracy: 0.8131 - val_loss: 0.4327
Epoch 3/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8186 - loss: 0.4225 - val_accuracy: 0.8194 - val_loss: 0.4240
Epoch 4/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8284 - loss: 0.4096 - val_accuracy: 0.8275 - val_loss: 0.4104
Epoch 5/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8336 - loss: 0.3977 - val_accuracy: 0.8331 - val_loss: 0.4000
Epoch 6/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8391 - loss: 0.3859 - val_accuracy: 0.8400 - val_loss: 0.3901
Epoch 7/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8464 - loss: 0.3754 - val_accuracy: 0.8419 - val_loss: 0.3799
Epoch 8/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8480 - loss: 0.3665 - val_accuracy: 0.8394 - val_loss: 0.3717
```

```
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8662 - loss: 0.3234 - val_accuracy: 0.8531 - val_loss: 0.3469
Epoch 46/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8656 - loss: 0.3226 - val_accuracy: 0.8562 - val_loss: 0.3479
Epoch 47/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8641 - loss: 0.3222 - val_accuracy: 0.8544 - val_loss: 0.3496
Epoch 48/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8678 - loss: 0.3223 - val_accuracy: 0.8544 - val_loss: 0.3487
Epoch 49/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8658 - loss: 0.3219 - val_accuracy: 0.8544 - val_loss: 0.3484
Epoch 50/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8653 - loss: 0.3217 - val_accuracy: 0.8550 - val_loss: 0.3464
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8610 - loss: 0.3403
Test Accuracy: 0.8610000014305115
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Confusion Matrix:
 [[1530   77]
 [ 201  192]]
```



9. Conclusion

The ANN model effectively predicts customer churn with high accuracy (~86%). The model generalizes well without overfitting and demonstrates the capability of neural networks in classification problems.